

APUNTESWEB · INGENIERÍA INFORMÁTICA



Tercer curso · 1.º cuatrimestre · 8 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción

NOTA

Esta asignatura es una locura la puta mierda apuntes que tiene, menos mal que en prácticas nos salvan. Me molaría reestructurar todo para que tuviese un orden un poco más lógico, pero me da pereza.

1.1 Definición general

1.1 Definición General: ¿De qué va esto?

Definición Formal

La **Teoría de Autómatas y Lenguajes Formales** estudia modelos matemáticos para representar procesos de cálculo. Su objetivo es entender qué problemas pueden resolver los ordenadores y con qué eficiencia. Aborda dos preguntas clave:

- **Decidibilidad:** ¿Qué puede hacer un ordenador?
- **Complejidad:** ¿Con qué eficiencia puede hacerlo?

Nota del Tutor (El concepto real)

Piensa en esta asignatura como el "**Diseño de Máquinas que leen cosas**". Todo se reduce a un esquema básico:

1. **Input:** Le das una cadena de texto a la máquina.
2. **Proceso:** La máquina (el autómata) procesa símbolo a símbolo.
3. **Output:** La máquina responde **SÍ** (Aceptado/Válido) o **NO** (Rechazado/Inválido).

1.2 El Ecosistema de la Teoría de Autómatas

1.1 El "Trío Sagrado": Lenguaje, Gramática y Máquina

Imagina que quieres preparar un plato específico. Necesitas tres elementos distintos pero conectados. Aquí veremos qué es cada uno y dónde se usan en la vida real:

1. El Lenguaje (El Plato Final)

Es el concepto abstracto. Es el **conjunto de cadenas (palabras)** que queremos aceptar como válidas.

- **Definición:** El objetivo final.
- **Ejemplo:** "El conjunto de todos los emails válidos" o "Todo el código C++ que funciona".

2. La Gramática (La Receta) [Sintaxis]

Son las **reglas generativas**. Te dicen cómo construir una cadena válida paso a paso. Es el ente "constructor".

- **Función:** Definir la estructura correcta.
- **Uso real: Compiladores.** Cuando VS Code te marca un error en rojo, es porque tu código rompió las reglas de la gramática del lenguaje.

3. El Autómata (El Crítico de Comida) [Reconocedor]

Es la **máquina que verifica**. Le das una cadena y te dice "Sí, pertenece al lenguaje" o "No, rechazada". Es el motor de búsqueda.

- **Función:** Buscar patrones y validar.
- **Uso real: Ctrl+F** en Word, validación de formularios web o el comando `grep` en Linux (usando Expresiones Regulares para búsquedas compactas).

LA RELACIÓN FUNDAMENTAL:

Para cada tipo de Lenguaje, existe una Gramática que lo genera y una Máquina que lo reconoce. Son tres caras de la misma moneda.

1.3 El "Diccionario" de la Asignatura

Peligro de Examen: Confundir estos términos es la causa #1 de suspensos

Concepto	Símbolo	Definición Formal	Analogía Práctica
Alfabeto	Σ	Conjunto finito y no vacío de símbolos (ej: $\{0, 1\}$).	Las piezas de Lego disponibles.
Palabra	w, x, y	Secuencia finita de símbolos del alfabeto.	Una torre construida con esas piezas.
Longitud	$ w $	Número de símbolos de w .	La altura de esa torre (nº de piezas).
Lenguaje	L	Conjunto de palabras ($L \subseteq \Sigma^*$).	La foto de la colección de torres válidas.

1.4 Conceptos Críticos y Operaciones

1.4.1 El error más común: Cadena Vacía vs. Lenguaje Vacío

Es fundamental distinguir entre "tener una caja vacía" y "tener una caja con una hoja en blanco dentro".

1. Cadena Vacía (ε o λ):

- Es una palabra que existe, pero **no tiene símbolos**.
- Su longitud es 0 ($|\varepsilon| = 0$).
- *Analogía*: Un string vacío en programación `""`.

2. Lenguaje Vacío ($\emptyset$):

- Es un conjunto que **no contiene ninguna palabra** (ni siquiera la vacía).
- *Analogía*: Una carpeta de archivos vacía.

Regla de oro:

- $L = \{\varepsilon\} \rightarrow$ Un lenguaje que contiene una palabra (la vacía). **NO está vacío**.
- $L = \emptyset \rightarrow$ Un lenguaje sin palabras. **ESTÁ vacío**.

1.4.2 Operaciones Básicas

Al igual que sumamos números, aquí operamos con palabras y lenguajes.

Sobre Palabras (Cadenas):

- **Concatenación** (xy): Pegar y detrás de x . **OJO**: El orden importa ($ab \neq ba$).
- **Potencia** (x^i): Repetir la cadena i veces. (Ej: $a^3 = aaa$).
- **Reflexión / Inversa** (x^R o x^{-1}): Leerla al revés (de derecha a izquierda).

Sobre Lenguajes (Conjuntos):

- **Concatenación** ($L_1 \cdot L_2$): Combina *cada* palabra del primer lenguaje con *cada* palabra del segundo.
- **Cierre de Kleene / Estrella** (L^*): La operación más importante.
 - Representa repetir palabras del lenguaje **cero o más veces**.
 - **Siempre** incluye la cadena vacía ε .
 - Σ^* : Significa "El conjunto de **todas** las palabras posibles que se pueden formar con el alfabeto".

1.4.3 Determinismo

El determinismo es **previsibilidad absoluta**. En un sistema determinista, si conoces el estado actual y la entrada que llega, sabes con certeza matemática qué va a pasar después. No hay dudas, no hay elecciones.

El no determinismo es la capacidad de **elegir o explorar múltiples futuros a la vez**. Ante una misma situación, la máquina puede tener varias opciones válidas de movimiento (o ninguna).

1.5 La Jerarquía de Chomsky: Las "Muñecas Rusas"

Aquí es donde entra el lío de los nombres. No son categorías aisladas, son **niveles de complejidad** concéntricos.

- **Regla de Oro:** Cada nivel **incluye** a todos los anteriores (es un subconjunto estricto).
- *Ejemplo:* Todo lo que es Regular (Nivel 3) es TAMBIÉN Independiente del Contexto (Nivel 2), Sensible al Contexto (Nivel 1) y Rec. Enumerable (Nivel 0).

Vamos del más simple (restrictivo) al más potente (libre).

Nivel 3: Lo Regular (Sin Memoria)

Aquí la máquina es muy tonta. No tiene memoria auxiliar, solo sabe "en qué estado está ahora mismo".

- **Lenguaje:** Regular.
- **Gramática:** Regular (Lineal por la derecha o izquierda). Reglas muy rígidas (Lineal por la derecha o izquierda). Reglas muy rígidas ($A \rightarrow aB$ o $A \rightarrow a$).
- **Máquina: Autómata Finito (AFD / AFN).**
 - **¿Determinismo? EQUIVALENTE.** Da igual si es determinista o no, tienen la misma potencia.
 - **Ejemplo Real:** Validar un email, buscar con `Ctrl+F`.
 - **Ejemplo Matemático:** a^* (cualquier número de 'a'), o números de dos cifras.
 - **Limitación:** No sabe contar indefinidamente (no distingue $a^n b^n$).

Nivel 2: Independiente del Contexto (Memoria de Pila)

Aquí añadimos una memoria tipo "pila" (LIFO - Last In, First Out). Podemos guardar cosas, pero solo podemos leer la que está arriba del todo.

- **Lenguaje:** Independiente del Contexto (LIC).
- **Gramática:** Independiente del Contexto (GIC). Reglas tipo $A \rightarrow \alpha$ (una variable cambia por cualquier cosa).
- **Máquina: Autómata con Pila (AP).**
 - **Determinismo:** Aquí **NO** son equivalentes.

- El **AP No Determinista** es el "jefe" del Nivel 2. Reconoce **toda** la clase de los **Lenguajes Independientes del Contexto (LIC)**.
- El **AP Determinista** es menos potente (reconoce un subconjunto menor). Reconoce un **subconjunto estricto**: los **Lenguajes Independientes del Contexto Deterministas**.
 - **Ejemplo Real**: La sintaxis de lenguajes de programación (Java, C, Python), HTML (etiquetas que se abren y cierran).
 - **Ejemplo Matemático**: $a^n b^n$ (mismo número de a's que de b's).
 - **Limitación**: No puede comparar tres cosas a la vez ($a^n b^n c^n$).

Nivel 1: Sensible al Contexto (Memoria Acotada)

Aquí la memoria es una cinta, podemos movernos y reescribir, pero **no podemos usar más espacio del que ocupa la palabra de entrada**.

- **Lenguaje**: Sensible al Contexto.
- **Gramática**: Sensible al Contexto (GSC). Reglas donde importa qué hay a los lados ($xAy \rightarrow xBy$).
- **Máquina: Autómata Linealmente Acotado (ALA/LBA)**.
 - **Concepto clave**: Es una MT con cinta limitada por muros a izquierda y derecha.
 - **Ejemplo Real**: El lenguaje natural (la concordancia gramatical compleja en español o inglés).
 - **Ejemplo Matemático**: $a^n b^n c^n$ (tres conteos coordinados).

NOTA

- **ALA No Determinista**: Reconoce los **Lenguajes Sensibles al Contexto** (Tipo 1 de Chomsky). Esta es la definición estándar que se suele usar.
- **ALA Determinista**: Reconoce los **Lenguajes Sensibles al Contexto Deterministas** (más limitados que los no deterministas).

Nivel 0: Recursivamente Enumerable (El poder total)

Aquí no hay límites. Memoria infinita. Si existe un algoritmo para calcularlo, está aquí.

- **Lenguaje**: Recursivamente Enumerable.
- **Gramática**: Irrestricida (Sin restricciones).
- **Máquina: Máquina de Turing (MT)**.
 - **¿Determinismo? EQUIVALENTE**. El no determinismo no añade potencia, solo velocidad teórica (P vs NP).
 - **El Gran Peligro**: La máquina podría **no detenerse nunca** (bucle infinito) si la palabra no es válida.

Dentro del Nivel 0, existe una subdivisión vital para aprobar las preguntas de Verdadero/Falso.

Tipo de Lenguaje	¿Qué hace la máquina si la palabra es VÁLIDA?	¿Qué hace si la palabra es INVÁLIDA?	¿Es seguro?
Recursivo (Decidible)	Para y dice SÍ .	Para y dice NO .	Sí, siempre responde.
Rec. Enumerable (Semi-decidible)	Para y dice SÍ .	Puede parar y decir NO... O quedarse en bucle infinito .	No, puede colgarse.

1.6 Tabla Resumen Definitiva (La "Chuleta")

Nivel Chomsky	Lenguaje	Máquina (Autómata)	Determinismo vs No Det.	Ejemplo Matemático Clave
Tipo 3	Regular	Autómata Finito	Equivalentes	a^*b^* (Patrones)
Tipo 2	Indep. Contexto	Autómata de Pila	DIFERENTES (El No-Det es más potente)	$a^n b^n$ (Pares, Palíndromos)
Tipo 1	Sensible Contexto	Linealmente Acotado	(Complejo, se asume No-Det)	$a^n b^n c^n$ (Tríos)
Tipo 0	Rec. Enumerable	Máquina de Turing	Equivalentes	Cualquier algoritmo

Tipo de Máquina	¿Es más potente la No Determinista?	Razón
Autómata Finito	NO (Son equivalentes)	Existe un algoritmo mecánico para convertir cualquier no determinista en uno determinista sin perder información.
Autómata de Pila	SÍ	El determinista está limitado (solo reconoce lenguajes sin ambigüedad). El no determinista reconoce todos los independientes del contexto.
Autómata Linealmente Acotado (ALA)	¿? (Problema Abierto / Se asume que SÍ)	En teoría es una incógnita matemática no resuelta. Pero en la asignatura, el No Determinista es el que define los Lenguajes Sensibles al Contexto (Tipo 1).
Máquina de Turing	NO (Son equivalentes)	Cualquier cálculo de una MT no determinista puede ser simulado por una determinista (aunque tarde más tiempo).
Complejidad (P vs NP)	SÍ (En tiempo)	Aunque resuelven lo mismo, la No Determinista lo hace en tiempo polinómico (rápido), mientras que la determinista podría tardar siglos.

2. Autómatas finitos

2.1 Autómata Finito Determinista (AFD)

Concepto Clave: "Una máquina sin dudas".

Un AFD es el modelo más estricto. Imagínalo como un tablero de juego donde, dado una casilla (estado) y una carta (símbolo), **solo tienes una única jugada legal**.

Características para identificar un AFD:

- **Determinismo:** Para cada estado y cada símbolo del alfabeto, existe **exactamente una** flecha de salida. Ni cero, ni dos. Una.
- **Sin magia:** No existen movimientos "gratuitos" (no hay transiciones ε).

Definición Formal (La "5-tupla"): En ejercicios teóricos, siempre debes definir estas 5 partes:

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q : Catálogo de todos los estados (los círculos).
- Σ : Alfabeto (las letras o números admitidos, ej: $\{0, 1\}$).
- δ : **El mapa de carreteras**. Es una función $Q \times \Sigma \rightarrow Q$. (Entra un estado y un símbolo, sale **un** estado).
- q_0 : Donde empieza todo (flecha sin origen).
- F : Donde ganamos (doble círculo).

Función de Transición Extendida ($\hat{\delta}$)

1. Diferencia Conceptual

- δ (**Transición simple**): Procesa **un solo símbolo** a la vez. Es el "paso a paso" del autómata.
- $\hat{\delta}$ (**Transición extendida**): Procesa una **cadena completa** (w) desde el estado actual hasta el final. Calcula la "ruta entera".

2. Definición Formal (Recursiva): Para procesar una cadena, la máquina descompone el problema por inducción:

- Caso Base (Cadena vacía): Si no leo nada, me quedo en el mismo estado.

$$\hat{\delta}(q, \lambda) = q$$

- Paso Inductivo (Cadena $w = xa$):

$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, x), a)$$

Significado: Primero calculo dónde acabo con el prefijo x (usando $\hat{\delta}$) y, desde ese estado intermedio, doy el último paso con el símbolo final a (usando δ).

3. **Definición de Lenguaje Aceptado:** Una cadena w pertenece al lenguaje del autómata $L(A)$ si, al procesarla entera desde el inicio, acabamos en un estado final:

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \in F\}$$

2.2 Autómata Finito No Determinista (AFN)

Concepto Clave: "Procesamiento en paralelo" o "Multiverso".

El AFN es una máquina teórica más flexible. A diferencia del AFD, aquí la máquina puede "adivinar" el camino correcto.

Diferencias prácticas para ejercicios

- **Ambigüedad:** Desde un estado con un símbolo (ej: 'a'), pueden salir **múltiples flechas o ninguna**.
- **Multiverso:** Si hay dos flechas con 'a', el autómata se clona y sigue ambos caminos a la vez.
- **Aceptación:** Si **al menos una** de las copias del autómata llega a un estado final al terminar la cadena, la cadena se acepta. Si todas mueren o acaban en no-finales, se rechaza.

Definición Formal La única diferencia real con el AFD está en la función de transición δ :

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$$

- **Traducción:** La función devuelve un **conjunto de estados** (potencia de Q), no un estado único. Puede devolver un conjunto vacío $\emptyset$ (callejón sin salida).

En el AFN, puedes estar en varios estados a la vez y elegir entre múltiples caminos.

Función de Transición Extendida en AFN ($\hat{\delta}$)

- **Concepto clave:** A diferencia del determinista (que devuelve *un estado*), en un AFN la función devuelve un **conjunto de estados** $\{p_1, \dots, p_k\}$, representando todos los caminos posibles simultáneos.
- **Definición Recursiva:**

1. **Base (λ):** Si no hay entrada, el conjunto es solo el estado actual: $\hat{\delta}(q, \lambda) = \{q\}$.

2. **Inducción** ($w = xa$): Para procesar una cadena, primero calculas los estados a los que llegas con el prefijo x , aplicas la transición de la última letra a a cada uno de ellos, y unes los resultados.

$$\hat{\delta}(q, w) = \bigcup \delta(p_i, a)$$

- **Condición de Aceptación:** Una cadena se acepta si, al terminar de leerla, el conjunto de estados posibles contiene al menos un estado final.

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

2.3 Transiciones ε (Epsilon) y Clausura- ε

Una transición ε es un **teletransporte**. Permite al autómata cambiar de estado **sin leer nada** de la cinta de entrada. Te permite estar en varios estados al mismo tiempo

Para resolver ejercicios de conversión, necesitas dominar la **Clausura- ε** .

- **Pregunta:** "¿A dónde puedo llegar desde aquí sin gastar ni una moneda (símbolo)?"
- **Regla:** La $\text{clausura-}\varepsilon(q)$ siempre incluye al propio estado q más cualquier estado alcanzable solo con flechas ε .

La clausura de la imagen anterior sería:

Algoritmo para sacar la clausura:

1. Sitúate en un estado (ej: q_0).
2. Mete q_0 en tu saco (siempre llegas a ti mismo).
3. Mira si salen flechas con ε . Si sí, sigue esas flechas a los nuevos estados.
4. Desde esos nuevos estados, ¿salen más ε ? Síguelas también.
5. Repite hasta que no puedas avanzar más sin leer letras.

Ejemplo: $q_0 \xrightarrow{\varepsilon} q_1 \xrightarrow{\varepsilon} q_2$

- $\text{Clausura}(q_0) = \{q_0, q_1, q_2\}$
- $\text{Clausura}(q_1) = \{q_1, q_2\}$

2.4 Equivalencia y Conversión: AFN $\rightarrow$ AFD

Los ordenadores reales no son "adivinos" (no son no-deterministas). Para programar un AFN, primero debemos convertirlo a AFD.

Como el AFN puede estar en varios sitios a la vez, **cada estado del nuevo AFD será un grupo de estados del AFN original.**

Para un ANF de n estados, el AFD equivalente tendrá como máximo 2^n estados.

Algoritmo:

1. **Inicio:** Calcula la *clausura- ε* (q_0) del AFN. Este conjunto de estados es tu estado inicial del AFD. Llámalo "A".
2. **Iteración:** Para el nuevo estado "A" y cada símbolo del alfabeto (ej: 0 y 1):
 - Mira a dónde van los estados dentro de "A" con ese símbolo.
 - A los destinos, aplícales *clausura- ε* .
 - El resultado es un nuevo conjunto. ¿Ya existe? Úsalo. ¿No existe? Bautízalo como "B".
3. **Iteración:** Repite el paso 2 con "B", "C", etc., hasta que no aparezcan conjuntos nuevos.
4. **Estados Finales:** Cualquier estado del AFD (A, B, C...) que contenga **al menos un** estado final del AFN original, se convierte en estado final.

Fórmula matemática (si la pide se la saca):

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$$

Imagina que estás en el estado combinado $S = \{q_1, q_2\}$ y llega una letra 'a'. ¿A dónde va la flecha?

1. Preguntas: "¿A dónde va q_1 con la 'a'?" → Digamos que va a $\{x, y\}$.
2. Preguntas: "¿A dónde va q_2 con la 'a'?" → Digamos que va a $\{z\}$.
3. **Haces la Unión ($\cup$):** El resultado es el nuevo estado $\{x, y, z\}$.

Ejemplo práctico:

2.5 Minimización de AFD

Objetivo: Encontrar el autómata más pequeño posible que haga exactamente lo mismo. Elimina redundancia.

Algoritmo

1. Partición Inicial (E_0): Divide los estados en solo dos grupos (clases):
 - **Finales (F):** Todos los estados de aceptación.
 - **No Finales ($Q \setminus F$):** El resto.
 - *Etiqueta cada grupo como $C_1, C_2, \dots$*

2. Construcción de la Tabla de Transiciones: Para cada estado, anota a qué Grupo (C_x) viaja con cada símbolo (0, 1...), basándote en la partición anterior.
 - *Truco*: No mires el estado destino, mira la **clase** del estado destino.
3. Refinamiento ($E_1, E_2 \dots$): Analiza los grupos formados:
 - Si dentro de un grupo (ej. C_1), todos los estados tienen el **mismo patrón de clases destino**, se quedan juntos.
 - Si un estado tiene un patrón diferente, se separa ("rompe" el grupo) y crea una nueva clase para la siguiente iteración.
4. Parada: Repite el proceso hasta que E_n sea idéntica a E_{n-1} (ya no se rompen más grupos).
5. Reconstrucción: Cada grupo final es un único estado en el autómata minimizado.

2.6 Equivalencia entre Estados

Dos estados p y q son equivalentes si son **indistinguibles** para un observador externo.

Imagínate que metemos el estado p en una caja negra y el estado q en otra. Tú no puedes ver el dibujo del autómata. Solo puedes meter cadenas de texto (inputs) y ver si la luz de "Aceptado" se enciende o no.

- Si para **cualquier** cadena infinita de pruebas que se te ocurra, las dos cajas siempre coinciden (o ambas aceptan, o ambas rechazan), entonces los estados son **equivalentes**.
- Si encuentras **una sola** cadena donde una caja dice "Sí" y la otra "No", entonces son **distinguidos** (no equivalentes).

Definición Formal

Sea un autómata (o dos) sobre un alfabeto Σ . Dos estados p y q son equivalentes (se denota $p \equiv q$) si y solo si:

$$\forall w \in \Sigma^* : (\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F)$$

Traducción: Para toda palabra w (desde la cadena vacía hasta una palabra de un millón de letras), si partimos de p y leemos w , el resultado final (aceptación o rechazo) debe ser exactamente el mismo que si partimos de q y leemos esa misma w .

Por lo que si nos dicen que si dos lenguajes regulares son equivalentes entre sí si los estados iniciales de sus correspondientes AFD son equivalentes

La equivalencia de estados es transitiva. Es decir, si para un AFD dos estados p y q son equivalentes y q y r son equivalentes, entonces p y r son equivalentes.

3. Lenguajes Regulares

Las **Expresiones Regulares (ER)** son una forma declarativa de describir lenguajes. Si el Autómata es la máquina que valida, la ER es la "fórmula" que genera las cadenas.

3.1 Operadores de las ER (Sintaxis y Jerarquía)

Para "leer" una Expresión Regular (ER) sin equivocarte, debes respetar la jerarquía. **Los 3 Operadores Fundamentales:**

1. **Cierre de Kleene / Estrella (*):** Repetición de 0 a ∞ veces.

◦ *Definición formal:*

$$L^* = \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \dots$$

◦ *En cristiano:* $\{\epsilon, L, LL, LLL, \dots\}$

2. **Concatenación ($\cdot$ o implícito):** Secuenciación.

◦ LM : Una cadena de L pegada a una de M .

3. **Unión ($+$ ó $|$):** Selección (O lógica).

◦ $L + M$: Cadenas que pertenecen a L o pertenecen a M .

WARNING

Jerarquía de Precedencia (¡Memoriza esto!)

El orden de evaluación estricto es:

1. ***** (**Estrella**): Lo más fuerte. Se pega a lo que tiene inmediatamente a la izquierda.
2. **$\cdot$** (**Concatenación**): Lo siguiente en fuerza.
3. **+** (**Unión**): Lo más débil. Separa la expresión en bloques grandes.

Ejemplo de examen: $a + bc^*$

- ¿Qué se repite? Solo la c .
- Luego se concatena b con c^* .
- Al final, tienes dos opciones: o la cadena a , o la cadena formada por bc^* .

3.2 Construcción de ER (Definición Inductiva)

En teoría te pueden preguntar: "Defina formalmente una ER". No te inventes nada, usa esta estructura recursiva:

1. Base (Los ladrillos):

- ε es una ER (representa el lenguaje $\{\varepsilon\}$).
- $\emptyset$ es una ER (representa el lenguaje vacío $\{\}$).
- Cualquier símbolo a del alfabeto es una ER (representa $\{a\}$).

2. Paso Inductivo (El cemento):

- Si E y F son ER, entonces también lo son:
 - $E + F$ (Unión)
 - EF (Concatenación)
 - E^* (Cierre)
 - (E) (Paréntesis para agrupar)

3.3 El "Cheat Sheet" del Álgebra de ER

Usa estas identidades para simplificar expresiones monstruosas en los ejercicios.

A. Identidades y Elementos Nulos

- Identidad de la Unión ($\emptyset$): $L + \emptyset = L$ (Sumar nada, se queda igual).
- Identidad de la Concatenación (ε):
 $L\varepsilon = \varepsilon L = L$ (Pegar "vacío" no alarga la cadena).
- Elemento Nulo de la Concatenación ($\emptyset$):
 $L\emptyset = \emptyset L = \emptyset$ (Si un tramo del puente se cae, no cruzas el río).

B. Propiedades Aritméticas

- **Conmutativa (SOLO Unión):** $L + M = M + L$.
 - ¡OJO!: La concatenación **NO** es conmutativa ($LM \neq ML$). "Casa" $\neq$ "Saca".
- **Asociativa:**
 - $(L + M) + N = L + (M + N)$
 - $(LM)N = L(MN)$
- **Distributiva (Factorización):** Vital para sacar factor común.

- Por izquierda: $L(M + N) = LM + LN$
- Por derecha: $(M + N)L = ML + NL$

- **Idempotencia:**

- $L + L = L$ (No sumas cantidades, unes conjuntos. Decir "rojo o rojo" es "rojo").

C. Propiedades de los Cierres (Estrella y Más)

Aquí es donde suelen pillar en los test.

1. **Doble Estrella:** $(L^*)^* = L^*$ (Repetir repeticiones sigue siendo repetir).

2. **Cierre Positivo (+):** $L^+ = LL^*$.

- Significa "1 o más veces" (excluye ε si L no lo contenía).

3. **Opcionalidad (?):** $L? = L + \varepsilon$.

- Significa "0 o 1 vez" (aparece o no aparece).

4. **Descomposición:** $L^* = L^+ + \varepsilon$.

5. **Casos Límite:**

- $\emptyset^* = \varepsilon$ (El conjunto de 0 repeticiones de nada es la cadena vacía).
- $\varepsilon^* = \varepsilon$

¿POR QUÉ $\emptyset^* = \varepsilon$?

La operación estrella (*) se define como la unión de todas las potencias de un lenguaje, empezando obligatoriamente por el cero:

$$L^* = L^0 \cup L^1 \cup L^2 \dots$$

1. **La Regla de Oro:** Por definición universal, **cualquier lenguaje** elevado a la potencia 0 es $\{\varepsilon\}$ (la cadena vacía).

2. **El Resto:** Como no puedes sacar símbolos de un conjunto vacío, $\emptyset^1, \emptyset^2 \dots$ son todos $\emptyset$.

La Suma: $\{\varepsilon\} \cup \emptyset \cup \emptyset \dots = \{\varepsilon\}$

3.4 Trampas Típicas de Examen

1. **Confundir $\emptyset$ con ε :**

- ε es una cadena (longitud 0). Es como una caja vacía.

- $\emptyset$ es un lenguaje (tamaño 0). Es no tener ni caja.
- Recuerda: $\emptyset^* = \varepsilon$.

2. El error de la distribución del asterisco:

- $(a + b)^* \neq a^* + b^*$.
- $(a + b)^*$ mezcla as y bs libremente (ej: $abaabb$).
- $a^* + b^*$ te obliga a elegir: o todo as, o todo bs.

3. Olvidar el orden en la concatenación:

- Si tienes $ab + ac$, puedes sacar factor común $a(b + c)$.
- Pero si tienes $ba + ca$, el factor común va a la derecha: $(b + c)a$.

3.5 Conversión de Autómatas Finitos a ER

Método: Eliminación de Estados.

La idea es desmantelar el autómata estado por estado hasta que solo quede una "super-flecha" del inicio al final con la Expresión Regular completa.

Algoritmo:

1. **Limpieza:** Elimina los **estados sumideros** (o "de muerte") que no llegan a ningún lado. Al quitarlo, debes "recablear" las conexiones para no perder información (ver la "Regla del Puente" abajo). **Si son finales no los borres.**
2. Resultado Final:
 - Si el estado inicial es también final: Te quedará 1 solo estado.
 - Si son distintos: Te quedarán 2 estados.
 - *Nota:* Si hay múltiples estados finales, calcula la ER para cada uno por separado (ignorando que los otros son finales) y únelas con un $+$ al final.

Regla del puente

Cuando eliminas un estado intermedio (q), cualquier camino que pasaba por él debe convertirse en una flecha directa.

Si tienes: $A \rightarrow q \rightarrow B$

- Y q tiene un bucle sobre sí mismo (K).
- La nueva flecha directa $A \rightarrow B$ será:

$$Etiqueta_{nueva} = (Entrada) \cdot (Bucle)^* \cdot (Salida)$$

- Si ya existía una flecha directa de A a B con valor *Directo*, se suma:

$$Total = Directo + (Entrada \cdot Bucle^* \cdot Salida)$$

1 Solo estado (Inicial = Final)

Si el autómata se reduce a un solo estado con uno o varios bucles.

$$L = R^*$$

R: La unión de todas las expresiones de los bucles en el estado ($r_1 + r_2 + \dots$).

2 Estados (Inicial $\neq$ Final)

Te queda el estado Inicial (q_0) y el Final (q_f), con flechas de ida, vuelta y bucles propios.

$$L = (R^* + SU^*T)^* SU^*$$

- $(R^* + SU^*T)^*$: Es todo lo que puedes hacer empezando y acabando en el inicio.
 - O giras en el inicio (R).
 - O vas al final, giras allí y vuelves ($S \cdot U^* \cdot T$).
 - Todo esto repetido las veces que quieras (*).
- SU^* : Una vez te cansas de dar vueltas en el inicio, **viajas al final (S)** y puedes quedarte girando allí (U^*) para terminar.

Esto es equivalente a escribir:

$$L = (R + SU^*T)^* SU^*$$

Porque podemos aplicar esta propiedad para simplificar: $(L^*)^* = L^*$

Ejemplo complejo

NOTA

Cuando hice el ejercicio de arriba se me fue la pinza. En ER_1 donde pone un $+$ es una multiplicación y lo mismo en ER_2 . $ER_1 = (1 + (00^* 10^* 11^* 0))^* (00^* 1^*)0^*$.

3.6 Conversión de ER a Autómatas Finitos

Empleando estas reglas se puede construir un AFD con transiciones epsilon, suelen quedar autómatas gigantescos. Se puede simplificar después o también hay casos donde es obvio el autómata que reconocen

$$R + S : L(R) + L(S)$$

$$RS : L(R)(S)$$

$$R^* : L(R^*)$$

Ejemplo:

3.7 Lema del Bombeo para Lenguajes Regulares

NOTA

Esto en prácticas no lo dimos, y ns si cae en el final o no pero en anteriores finales lo pregunta. Y claro no se si en la teórica lo dijo porque ir a la teórica nunca fue opción.

Sea L un lenguaje regular. Entonces existe un número entero $p \geq 1$ tal que cualquier cadena w perteneciente a L con longitud $|w| \geq p$ puede dividirse en tres partes, $w = xyz$, cumpliendo las siguientes tres condiciones:

- $|y| > 0$ (o lo que es lo mismo, $y \neq \epsilon$): La parte central no puede estar vacía.
- $|xy| \leq p$: La parte que se repite ocurre dentro de los primeros p caracteres.
- **Para todo $k \geq 0$, la cadena $xy^kz \in L$:** Podemos repetir la parte y tantas veces como queramos (o borrarla con $k = 0$) y la cadena resultante seguirá perteneciendo al lenguaje.

Podemos pensarlo como:

- x (**El prefijo**): El camino desde el inicio hasta antes de entrar al bucle.
- y (**El bucle**): La parte de la cadena que hace que el autómata de una vuelta y regrese al mismo estado. Por eso puedes repetirla (k veces) y el autómata sigue feliz en ese bucle.
- z (**El sufijo**): El camino desde que sales del bucle hasta el estado final.

El Lema del Bombeo no garantiza que el lenguaje sea regular, porque aunque lo cumpla podría no serlo. Lo que es seguro es que si no lo cumple, no es regular.

Ejemplo: $L = \{a^i b^j \mid j = 2i\}$. Eso implica que $N_b = 2 \cdot N_a$ y a mayores se debe de respetar el orden. Normalmente nos darán algo así:

- $x = a^{(n/2)-1}$ (Aporta $\frac{n}{2} - 1$ aes, 0 bes).
- $y = abb$ (Aporta 1 a, 2 bes).
- $z = b^{n-2}$ (Aporta 0 aes, $n - 2$ bes).

Y nos dirán que probemos con varios k si se cumple o no. Si por ejemplo usamos $k = 2$

$$w' = a^{(n/2)-1} \cdot (abb) \cdot (abb) \cdot b^{n-2}$$

Ya podemos apreciar simple vista que va a fallar porque tenemos $abbabb$ lo cual no cumple el orden, por ello no tenemos un lenguaje regular. Pero si nos dicen que $k = 0$ tendríamos

$$w' = x \cdot z$$

$$w' = a^{(n/2)-1} \cdot b^{n-2}$$

Y aprovechamos $N_b = 2 \cdot N_a$ para sustituir:

$$2 \cdot \left(\frac{n}{2} - 1\right) = n - 2$$

Vemos que el lema del bombeo no falla, pero esto no significa que sea regular.

4. Gramáticas Independientes del Contexto

Una **gramática** es un conjunto de reglas que nos permite generar todas las palabras válidas de un lenguaje. Es como un "recetario" para crear frases correctas.

4.1 Clasificación de Gramáticas

Una gramática se define como: $G = (NT, T, P, S)$.

- **NT (o V):** Variables no terminales (Mayúsculas, ej: A, B).
- **T :** Terminales (minúsculas, ej: a, b).
- **P :** Producciones (Reglas Izquierda $\rightarrow$ Derecha).
- **S :** Axioma inicial.

Chomsky clasificó las gramáticas en 4 niveles (del 0 al 3). **La regla de oro:** A mayor número, más restricciones tiene la gramática y menos potente es.

4.1.1 Tipo 0: No Restringida (Recursivamente Enumerable)

- **Definición:** $x \rightarrow y$
 - $x \in (NT/T)^+$ (La izquierda debe tener *algo*, no puede ser vacío).
 - $y \in (NT/T)^*$ (La derecha puede ser cualquier cosa o vacío).
- **Explicación Práctica:**
 - En la izquierda puedes tener mezclas de variables y terminales (ej: $aAb \rightarrow \dots$).
 - Puedes borrar símbolos, añadir cosas de la nada o acortar la cadena.
 - Esta gramática genera **Lenguajes Recursivamente Enumerables** y son reconocidos por la **Máquina de Turing**

4.1.2 Tipo 1: Sensible al Contexto

- **Definición:** $\alpha \rightarrow \beta$ tal que $|\alpha| \leq |\beta|$
 - Formalmente se ve así: $\alpha = z_1 \mathbf{x} z_2$ y $\beta = z_1 \mathbf{y} z_2$.
 - Significa que la variable $\mathbf{x}$ se transforma en $\mathbf{y}$ solo si está rodeada por el contexto z_1 y z_2 .
 - $z_1, z_2 \in T^*$, $x \in NT$ e $y \in (NT/T)^+$
- **Explicación Práctica:**
 - La longitud de lo de la derecha (β) siempre es **mayor o igual** que lo de la izquierda (α). Nunca se acorta la cadena (excepto quizás para borrar el axioma inicial).

- Necesitas "vecinos" específicos para activar el cambio.
- Esta gramática genera **Lenguajes Sensibles al Contexto** y son reconocidos por el **Autómata Linealmente Acotado** (una versión de la Máquina de Turing que tiene una cinta de memoria finita/limitada)

4.1.3 Tipo 2: Independiente del Contexto (GIC / CFG)

- **Definición:** $x \rightarrow y$
 - $x \in NT$ (¡OJO! La izquierda **siempre** es una sola Variable/Mayúscula).
 - $y \in (NT/T)^*$ (La derecha es cualquier combinación).
- **Explicación Práctica:**
 - Es una **sustitución simple**. No importa qué haya al lado de la variable, si ves una A , puedes cambiarla por su regla.
 - Ejemplo: $A \rightarrow aBc$.
 - Esta gramática genera **Lenguajes Independientes del Contexto** y son reconocidos por el **Autómata con Pila (PDA)**.

4.1.4 Tipo 3: Regular

- **Definición:** $\alpha \rightarrow \beta$
 - $\alpha \in NT$ (Izquierda es una sola Variable).
 - β tiene varias formas:
 - $\beta \in aB$
 - $\beta \in Ba$
 - $\beta \in b$
 - $B \in NT$
 - $a \in T^+$
 - $b \in T^*$
- **Explicación Práctica:**
 - Es una "cola india". La cadena va creciendo linealmente.
 - Se divide en dos subtipos (que **NO** se pueden mezclar en la misma gramática):
 1. **Lineal por la Derecha:** $A \rightarrow aB$ o $A \rightarrow a$. (La variable siempre al final).
 2. **Lineal por la Izquierda:** $A \rightarrow Ba$ o $A \rightarrow a$. (La variable siempre al principio).
 - Esta gramática genera **Lenguajes Regulares** y son reconocidos por los **Autómatas Finitos** (DFA o NFA), que son máquinas sin memoria auxiliar, limitadas a reconocer patrones lineales.

Relación de Conjuntos:

$$T_3 \subset T_2 \subset T_1 \subset T_0$$

4.4 Lenguaje de una Gramática Independiente del Contexto

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$$

Se lee así: "El lenguaje L generado por la gramática G es el conjunto de cadenas w ..."

1. $w \in T^*$:

- Esto impone la condición principal: la cadena resultante w debe estar formada **únicamente por símbolos Terminales** (T).
- El asterisco (*) es el **Cierre de Kleene**, que significa "cualquier combinación de terminales".
- **¿Por qué es importante?** Porque durante la derivación intermedia puedes tener cosas como aSb (que mezcla terminales y variables). La fórmula te dice que eso **no** es parte del lenguaje final todavía; solo lo es cuando desaparecen todas las letras mayúsculas (Variables).

2. $\mid$:

- Significa "tal que" o "que cumplen la condición de que...".

3. $S \Rightarrow^* w$:

- S : Debes empezar obligatoriamente desde el **Símbolo Inicial**.
- $\Rightarrow^*$: La flecha doble con asterisco significa "**se deriva en cero o más pasos**". Es decir, no importa si tardas 1 paso o 1.000 pasos en llegar.
- w : Debes llegar a la palabra final.

Ejemplo:

$S \rightarrow aSb \mid \epsilon$
 $T = \{a, b\}$

Derivaciones:

- $S \Rightarrow \epsilon \rightarrow$ palabra: "" (cadena vacía)
- $S \Rightarrow aSb \Rightarrow ab \rightarrow$ palabra: "ab"
- $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb \rightarrow$ palabra: "aabb"
- $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaSbbb \Rightarrow aaabbb \rightarrow$ palabra: "aaabbb"

$$L(G) = \{a^n b^n \mid n \geq 0\} = \{\epsilon, ab, aabb, aaabbb, \dots\}$$

4.5 Árboles de Derivación vs Derivaciones

Derivar es el proceso de construir una cadena de texto válida (una palabra) aplicando las reglas de la gramática paso a paso.

Se empieza con el **Símbolo Inicial** (generalmente llamado S) y se van sustituyendo las variables (letras mayúsculas) por lo que dictan las reglas (producciones) hasta que solo quedan **terminales** (letras minúsculas o símbolos finales que ya no se pueden cambiar).

Es básicamente un juego de "**buscar y reemplazar**":

- **Entrada:** El símbolo inicial S .
- **Acción:** Eliges una variable presente en tu cadena actual, buscas una regla que le aplique y la sustituyes.
- **Fin:** Cuando ya no quedan variables (letras mayúsculas), has terminado la derivación.

Es importante distinguir entre el **proceso** y la **estructura**.

1. **Derivación (Texto):** Es la secuencia de pasos paso a paso.

$$\circ S \Rightarrow aA \Rightarrow abB \Rightarrow abb$$

2. **Árbol de Derivación (Gráfico):** Es la estructura jerárquica.

- Raíz: S .
- Hojas: La palabra final (a, b, b).
- Nodos: Variables intermedias.

Gramática:

1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow id$ (donde 'id' es un número o variable)

Objetivo: Generar el árbol para la cadena: $id + id * id$

Empezamos con el símbolo inicial E .

Miramos nuestra cadena objetivo ($id + id * id$). Vemos que hay una suma principal (o una multiplicación, depende de cómo decidamos estructurarlo, pero asumamos la suma primero para este ejemplo).

Aplicamos la regla $E \rightarrow E + E$.

- De la raíz E salen tres hijos: E , $+$ y E .

Ahora tenemos:

- Un E a la izquierda.

- Un $+$ en el medio (ya es terminal, se queda quieto).
- Un E a la derecha.

Miramos la cadena objetivo: la primera parte es solo id . Así que al E de la izquierda le aplicamos la regla $E \rightarrow id$.

Para la segunda parte, necesitamos $id * id$. Así que al E de la derecha no lo convertimos en id directamente, sino que le aplicamos la regla de multiplicación: $E \rightarrow E * E$

Ahora el árbol ha crecido. Los nuevos E que creamos para la multiplicación se convierten cada uno en id usando la regla $E \rightarrow id$.

4.6 Ambigüedad

Definición de Examen: Una gramática es ambigua si existe **al menos una cadena** que tiene **dos o más árboles de derivación distintos**. Pueden tener derivaciones distintas y no ser ambiguas, por ejemplo:

1. $S \rightarrow AB$
2. $A \rightarrow a$
3. $B \rightarrow b$

Queremos generar la cadena: ab Podemos hacerlo con dos "derivaciones" diferentes (cambiando el orden en que sustituimos las letras):

Derivación 1 (Por la izquierda):

1. $S \Rightarrow AB$
2. $S \Rightarrow aB$ (Sustituyo A primero)
3. $S \Rightarrow ab$ (Sustituyo B después)

Derivación 2 (Por la derecha):

1. $S \Rightarrow AB$
2. $S \Rightarrow Ab$ (Sustituyo B primero)
3. $S \Rightarrow ab$ (Sustituyo A después)

¿Son derivaciones diferentes? Sí. La lista de pasos es distinta. En una escribí la 'a' antes y en la otra escribí la 'b' antes.

¿Son árboles diferentes? NO. Si dibujas el árbol, el resultado es idéntico en ambos casos:

- La raíz es S .
- De S salen dos ramas: A y B .
- De A cuelga una a .
- De B cuelga una b .

Caso típico: Operaciones matemáticas sin paréntesis ni precedencia.

- $3 + 4 * 5 \rightarrow$ ¿Es $(3+4)*5$ o $3+(4*5)$? Si la gramática permite ambos árboles, es mala (ambigua).

4.7 Protips para ejercicios

Patrón 1: El Espejo y la Cebolla (Palíndromos y $a^n b^n$)

Si necesitas que el principio coincida con el final, o que la cantidad de letras del principio sea igual a la del final, usas la **recursividad envolvente**.

La Regla de Oro:

$$S \rightarrow x S y$$

Esto genera x a la izquierda y y a la derecha, sincronizados.

Ejemplo Práctico Palíndromo:

1. Si añado una 'a' al principio, debo añadir una 'a' al final: $S \rightarrow aSa$
2. Si añado una 'b' al principio, debo añadir una 'b' al final: $S \rightarrow bSb$
3. ¿Cómo termino? Con el centro. Puede ser 'a', 'b', o vacío (ε): $S \rightarrow a \mid b \mid \varepsilon$

Gramática final: $S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon$

Patrón 2: Ecuaciones Lineales ($k = i + j$ o $k = i + 2j$)

Estos ejercicios te piden relacionar contadores.

- **Truco:** Traduce la ecuación a "quién consume a quién".
- **Orden:** Fíjate MUY bien en el orden de las letras en el alfabeto ($a^i b^j c^k$).

Caso A: Suma simple ($k = i + j$ en $a^i b^j c^k$) Significa que por cada 'a' hay una 'c', Y por cada 'b' hay una 'c'. Pero las 'a' están lejos de las 'c'.

- Solución: Anidamiento. Tratamos la cadena como $a^i (b^j c^j) c^i$.
- Las 'a' envuelven a todo el bloque de 'b' y 'c'.
- Las 'b' se emparejan con las 'c' del medio.

Caso B: Multiplicación ($k = i + 2j$ en $a^i b^j c^k$)

- Interpretación:
 - Por cada 1 'a', genero 1 'c'. (Exterior)

- Por cada 1 'b', genero 2 'c's. (Interior)

- Diseño:

1. Estado inicial (S): Genera 'a' izquierda y 'c' derecha. Cuando acaben las 'a', pasamos al bloque central (B).

$$S \rightarrow aSc \mid B$$

2. Estado central (B): Genera 'b' izquierda y dos 'c' derecha.

$$B \rightarrow bBcc \mid \varepsilon$$

Patrón 3: La "O" Lógica (Unión de Casos)

Si ves un "o", una coma, o condiciones alternativas ($i = j$ o $j = k$), **NO** intentes hacerlo todo en una sola regla. Divide y vencerás.

Estrategia: El símbolo inicial solo sirve para elegir camino.

$$S \rightarrow S_1 \mid S_2$$

Ejemplo ($a^i b^j c^k$ donde $i = j$ O $j = k$):

- Camino 1 (S_1): $i = j$ (y k va por libre). Cadena tipo $a^n b^n c^m$.
 - Empareja 'a' y 'b'. La 'c' se genera aparte libremente.
 - $S_1 \rightarrow XC$
 - $X \rightarrow aXb \mid \varepsilon$ (pareja a-b)
 - $C \rightarrow cC \mid \varepsilon$ (c libre)
- Camino 2 (S_2): $j = k$ (y i va por libre). Cadena tipo $a^m b^n c^n$.
 - La 'a' va libre al principio. Luego empareja 'b' y 'c'.
 - $S_2 \rightarrow AY$
 - $A \rightarrow aA \mid \varepsilon$ (a libre)
 - $Y \rightarrow bYc \mid \varepsilon$ (pareja b-c)

Patrón 4: Desigualdades (El Truco del "Sobra Algo")

Las gramáticas no saben hacer "mayor que". Solo saben hacer "igual". **Truco Matemático:**

- $k > i \rightarrow$ significa $k = i + m$ (donde $m \geq 1$).
- Es decir: "Hay tantas 'k' como 'i', y luego **sobran** más 'k'".

Ejemplo ($a^i (b + c)^k$ donde $k > i$): Vamos a simplificar $(b + c)$ llamándolo X . La estructura es $a^i X^k$.

1. Parte equilibrada: Por cada 'a', pongo una X .

2. Parte sobrante: Añado más X al final (o al lado de las X).

$$S \rightarrow aSX \mid A \quad (\text{Emparejo } a \text{ con } X)$$

$$A \rightarrow XA \mid X \quad (\text{Genero las } X \text{ sobrantes, al menos una})$$

$$X \rightarrow b \mid c \quad (\text{Defino qué es } X)$$

Truco para "Distinto" ($\neq$). "Distinto" significa "Mayor que" O "Menor que".

$$S \rightarrow S_{mayor} \mid S_{menor}$$

Haces dos gramáticas (como en el Patrón 4) y las unes.

Patrón 5: Desorden y Mezcla (Conteo $N(a) = N(b)$)

Aquí NO hay orden $a^i b^j$. Las letras pueden estar mezcladas "aababb...". Esto se resuelve con **Inserción Relativa**.

Regla Maestra para $N(a) = N(b)$: Si quiero mantener el equilibrio, donde ponga una 'a', debo poner una 'b'. Pero como no hay orden, la 'b' puede ir antes, después, o alrededor.

La forma estándar segura es:

$$S \rightarrow aSbS \mid bSaS \mid SS \mid \varepsilon$$

Significado: Si pongo 'a', debo "deber" una 'b' (el estado S intermedio se encarga de resolver esa deuda).

Ejemplo ($N(0) = N(1) + 1$): Esto es: "Equilibrado + un 1 extra".

- Definimos un equilibrio perfecto B (mismo n° de 0 y 1).

$$B \rightarrow 0B1B \mid 1B0B \mid \varepsilon$$

(Ojo: simplificado para el concepto, a veces se requiere más rigor con SS).

- Estado Inicial: Es el equilibrio B , pero forzando un '0' extra en algún sitio.

$$S \rightarrow B 0 B$$

4.7 Simplificación de Gramáticas (GIC)

Antes de pasar a formas normales, **siempre** debes limpiar la gramática en este orden estricto:

1. Eliminar Producciones ε (Vacías)

Si $A \rightarrow \varepsilon$, entonces A es "anulable". Método:

1. Busca todas las variables que pueden volverse ε (directa o indirectamente).
2. Si tienes $S \rightarrow Ab$, y A es anulable, añade una nueva regla $S \rightarrow b$ (versión donde A desaparece).
3. Borra las reglas $A \rightarrow \varepsilon$ originales (salvo si $S \rightarrow \varepsilon$ es necesario para el lenguaje)

2. Eliminar Producciones Unitarias ($A \rightarrow B$)

Reglas que solo cambian el nombre de la variable sin añadir terminales. Método:

1. Si $A \rightarrow B$ y $B \rightarrow \text{algo}$, entonces añade $A \rightarrow \text{algo}$.
2. Borra $A \rightarrow B$.

3. Eliminar Símbolos Inútiles

Se hace en dos pasadas:

1. **No Generadores:** Variables que entran en bucle y nunca llegan a terminales ($A \rightarrow aA$). Bórralas.
2. **Inalcanzables:** Variables a las que no puedes llegar empezando desde S . Bórralas.

Un símbolo X es generador si $X \Rightarrow *w$. Un símbolo X es alcanzable si existe una derivación $S \Rightarrow *\alpha X \beta$ para algún α y β . Todo símbolo útil es generador y alcanzableeeeeeeeeeeee.

Ejemplo que usa los pasos 1 y 2:

4.8 Formas Normales (FNC-Chomsky)

Objetivo: Estandarizar la gramática para que todas las reglas sean "cortas" y binarias. Es fundamental para el algoritmo CYK (análisis sintáctico).

Solo se permiten 2 tipos de reglas:

1. $A \rightarrow BC$ (Dos variables).
2. $A \rightarrow a$ (Un terminal).

Algoritmo de Conversión a FNC (Práctico)

Supongamos que ya has simplificado la gramática (paso 4.8).

Paso 1: Terminales solitarios en reglas mixtas. Si tienes $S \rightarrow aB$, eso está prohibido (mezcla terminal y variable).

- Crea una variable nueva: $X_a \rightarrow a$.
- Cambia la regla a: $S \rightarrow X_a B$.

Paso 2: Acortar cadenas largas. Si tienes $S \rightarrow ABC$ (3 variables), es demasiado largo.

- Rompe la cadena creando variables intermedias.

- $S \rightarrow AZ$
- $Z \rightarrow BC$

Ejemplo Rápido: Original: $S \rightarrow aSb$

1. Crear variables para terminales: $X_a \rightarrow a, X_b \rightarrow b$.
2. Sustituir: $S \rightarrow X_a S X_b$.
3. Romper cadena de 3:
 - $S \rightarrow X_a Y$
 - $Y \rightarrow S X_b$

Resultado FNC:

$$\begin{aligned} S &\rightarrow X_a Y \\ Y &\rightarrow S X_b \\ X_a &\rightarrow a \\ X_b &\rightarrow b \end{aligned}$$

4.9 Forma Normal de Greibach (FNG)

La **FNG** es otra forma de estandarizar gramáticas (como la de Chomsky), pero con una filosofía distinta: **"Producir letra a letra"**.

Imagina una máquina expendedora. En la **FNG**, cada vez que la gramática hace un movimiento (aplica una regla), está **obligada** a soltar exactamente **una moneda (símbolo terminal)** y quedarse con el cambio (variables).

La regla siempre tiene esta forma:

$$A \rightarrow a\alpha$$

- A : Variable actual.
- a : **Un solo terminal** (la moneda que suelta).
- α : Una cadena de cero o más variables (el cambio que te queda por procesar).

Ejemplo: $S \rightarrow aAB$ (Soltó una 'a', le queda procesar A y B).

Una cadena de longitud n tiene una derivación de n pasos. Un analizador sintáctico descendente parará a profundidad n y nunca habrá recursividad por la izquierda.

5. Autómatas con Pila

Concepto Intuitivo: Imagina un Autómata Finito (AFN) que lleva una mochila llena de platos.

- Puede leer la entrada.
- Puede **mirar** el plato de arriba de la pila.
- Dependiendo de lo que ve y lee, puede **quitar platos (pop)** o **poner platos nuevos (push)**.

Esta "mochila" (memoria LIFO) le permite contar y comparar, algo que un autómata finito simple no puede hacer (ej: saber si hay el mismo número de 'a' que de 'b').

5.1 Definición

Un autómata con pila (AP) es un AFN con transiciones ϵ y con una pila en la que se puede almacenar una cadena de símbolos de pila. El AP puede recordar una cantidad infinita de información. Reconoce Lenguajes Independientes del Contexto.

Un AP se define matemáticamente así:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Diferencias clave con el autómata finito:

1. Γ (**Alfabeto de Pila**): Símbolos que podemos guardar en la memoria. Puede ser diferente al de entrada (Σ).
2. Z_0 (**Fondo de Pila**): El símbolo que está en la pila antes de empezar nada. Nos avisa de que "la pila está vacía".
3. δ (Función de Transición): La "ley" del movimiento.

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$$

Lo que está antes de la flecha ($\rightarrow$) son las 3 condiciones que se tienen que cumplir para que la máquina pueda moverse. Significa: "Para hacer un movimiento, necesito saber..."

1. Q (**Estado actual**): ¿En qué círculo estoy ahora mismo? (Ej: estoy en q_0).
2. $\Sigma \cup \{\epsilon\}$ (**Lo que leo en la cinta**):
 - Σ : ¿Qué letra estoy leyendo de la palabra de entrada? (Ej: una 'a').
 - $\cup\{\epsilon\}$: O también puedo **no leer nada** (usar ϵ). Esto permite al autómata hacer movimientos "gratis" o espontáneos sin consumir letras de la entrada.
3. Γ (**Tope de la Pila**): ¿Qué símbolo está encima del todo en la pila? (Ej: hay una 'Z').

- *Nota:* A diferencia del autómata finito, aquí **es obligatorio** mirar y sacar (pop) el símbolo superior de la pila para decidir.

En resumen (Izquierda): "Estoy en el estado q , leo la letra a y saco de la pila el símbolo Z . Lo que está después de la flecha es la consecuencia. Significa: "Esto es lo que voy a hacer..."

1. $P(\dots)$ (**Partes de / Conjunto Potencia**):

- Esto es crucial. La P indica que el resultado es un **conjunto de opciones**, no una sola.
- Esto confirma que es un **Autómata NO Determinista**. Ante una misma situación, la función puede devolver varias parejas de (estado, pila) posibles.

2. Q (**Nuevo Estado**): ¿A qué estado me voy ahora? (Ej: me muevo a q_1).

3. Γ^* (**Lo que escribo en la Pila**):

- Aquí decides qué metes de vuelta en la pila. Como tiene el asterisco (*), significa que puedes escribir una cadena de **cualquier longitud** de símbolos de pila.

La parte Γ^* es la palanca de cambios de la memoria:

- **Si escribes ε (vacío):** Has sacado el tope para leerlo y no metes nada. → **Desapilar (POP)**
- **Si escribes el mismo símbolo que sacaste (ej: 'Z'):** Lo sacaste y lo volviste a meter igual. → **Quedarse igual (No-Op).**
- **Si escribes dos o más símbolos (ej: 'AZ'):** Sacaste 'Z' y metes 'AZ'. Has añadido 'A' encima. → **Apilar (PUSH).**

Imagina esta transición concreta derivada de tu fórmula:

$$\delta(q_0, a, Z) = \{(q_1, AZ)\}$$

Lectura:

1. **Situación:** Estoy en estado q_0 , leo una 'a' en la entrada y el tope de la pila es 'Z'.
2. **Acción:**
 - Cambio al estado q_1 .
 - Sustituyo la 'Z' que saqué por la cadena "AZ".
 - **Efecto:** He guardado (apilado) una A sobre la Z.

En este tema solo se habla de los APN, pero para preguntas tipo test hay que saber lo siguiente.

5.1.1 Autómata de Pila NO Determinista (APN)

Es el modelo estándar y el más potente de los dos.

- **Comportamiento:** Ante una misma situación (mismo estado, misma entrada y mismo símbolo en el tope de la pila), la máquina puede tener **varias opciones** de movimiento.
 - Puede elegir desapilar, no hacer nada, o apilar diferentes cosas.
 - La máquina "adivina" mágicamente cuál es el camino correcto para aceptar la cadena.

- **Lenguajes que reconoce:** Reconoce **todos** los **Lenguajes Independientes del Contexto (LIC)** (o Libres de Contexto).
 - *Ejemplo clásico:* Los palíndromos pares (ww^R , como "anitalavalatina"). La máquina necesita el no determinismo para "adivinar" cuándo ha llegado exactamente a la mitad de la palabra y debe empezar a comprobar la segunda parte con la pila.

5.1.2 Autómata de Pila Determinista (APD)

Es una versión restringida y **menos potente**.

- **Comportamiento:** Ante una situación dada (estado, entrada, tope de pila), existe **como máximo una** acción posible.
 - Nunca hay duda de qué hacer. Es predecible, como un programa de ordenador normal.
- **Lenguajes que reconoce:** Reconoce los **Lenguajes Independientes del Contexto Deterministas**.
 - Estos lenguajes están "a medio camino entre los lenguajes regulares y los independientes al contexto".
 - Son un subconjunto estricto. **No todos** los lenguajes libres de contexto pueden ser reconocidos por un APD.

Característica	AP NO Determinista (APN)	AP Determinista (APD)
Capacidad de elección	Múltiples transiciones posibles.	Solo una transición posible.
Potencia	Mayor. Reconoce toda la familia de lenguajes de contexto libre.	Menor. Solo reconoce un subconjunto.
Lenguajes	Lenguajes Independientes del Contexto (LIC).	Lenguajes Indep. del Contexto Deterministas .
Ejemplo de Lenguaje	Palíndromos (ww^R) sin marcar el centro.	Código de programación (C, Java), expresiones aritméticas.
Equivalencia	NO es equivalente al Determinista.	NO es equivalente al No Determinista.

5.2 Mecánica de Transición (Cómo leer los arcos)

Esta es la parte vital para los ejercicios prácticos. En los diagramas verás arcos con la etiqueta:

$$a, A \rightarrow \gamma$$

Esto se lee: "**Leo, Saco** $\rightarrow$ **Meto**".

1. **Leo (a):** Leo el símbolo 'a' de la cinta de entrada.

2. **Saco (A):** Compruebo si A está en la **cima** de la pila y lo extraigo (pop).
3. **Meto (γ):** Escribo la cadena γ en la cima de la pila (push).

Casos Prácticos de "Meto" (γ):

- **Apilar (Push):** $a, Z_0 \rightarrow AZ_0$
 - *Explicación:* Saco Z_0 y meto A seguido de Z_0 . Efecto neto: A queda encima de Z_0 .
- **Desapilar (Pop):** $a, A \rightarrow \varepsilon$
 - *Explicación:* Saco A y meto... nada (ε). Efecto neto: Borro A .
- **Cambiar (Swap):** $a, A \rightarrow B$
 - *Explicación:* Saco A y meto B .
- **No tocar (Peep):** $a, A \rightarrow A$
 - *Explicación:* Saco A y vuelvo a meter A . La pila se queda igual.

5.3 Tipos de Aceptación

Un AP puede decir "OK" de dos formas. En los ejercicios te especificarán cuál usar.

Aceptación por Estado Final (F)

- **Condición:** La entrada se ha terminado ($w = \varepsilon$) **Y** el autómata está en un estado $q \in F$.
- **La pila:** No importa lo que tenga dentro (puede estar llena de basura).
- **Uso:** Es lo más parecido a los autómatas normales.

El lenguaje aceptado por **estado final** se define como:

$$L(P) = \{w | (q_0, wZ_0) \vdash^* (q, \varepsilon, \alpha)\}$$

Dónde q pertenece al conjunto de estados finales

Aceptación por Pila Vacía ($\emptyset$)

- **Condición:** La entrada se ha terminado ($w = \varepsilon$) **Y** la pila está totalmente vacía (ni siquiera queda Z_0).
- **El estado:** No importa en qué estado termine.
- **Uso:** Muy común en análisis sintáctico (compiladores).

El lenguaje aceptado por **estado final** se define como:

$$L(P) = \{w | (q_0, wZ_0) \vdash^* (q, \epsilon, \epsilon)\}$$

Conversión: Todo lenguaje aceptado por pila vacía puede ser aceptado por estado final y viceversa. Son equivalentes en poder.

5.4 Protips

Patrón 1: "El Acumulador" (Sumar cosas)

Cuándo usarlo: Ecuaciones tipo $k > i + j$ o $k = i + j$. **Lógica:** Tienes dos variables que "suman" y una que "resta".

- **Fase 1 (Entrada 'a'):** Apilas 'a'.
- **Fase 2 (Entrada 'b'):** Sigues apilando (pero ojo, como tienes la restricción, apila 'b' o sigue apilando 'a' si te dejan. Aquí dice "alfabeto de pila = alfabeto entrada", así que apila 'b').
- **Fase 3 (Entrada 'c'):** Desapilas todo. Primero las 'b' y luego las 'a'.
- **Resultado:** Si al acabar de leer 'c' la pila se vacía, eran iguales. Si sobra pila, $i + j$ era mayor. Si falta pila, k era mayor.

Patrón 2: "La Deuda" (El orden da igual / Sopa de letras)

Cuándo usarlo: Ejercicios donde el orden es libre ($N(a) = N(b)$ entrando en cualquier orden) o ecuaciones complejas ($i + k = j + m$).

Lógica: La pila representa el Balance.

- Define dos bandos: **Positivos** (los que suman) y **Negativos** (los que restan).
- **Regla de Oro:**
 - Si leo un símbolo y la pila tiene al del **bando contrario**: **DESAPILO** (se cancelan, como materia y antimateria).
 - Si leo un símbolo y la pila tiene al de **mi bando** o está vacía (Z_0): **APILO** (aumento mi deuda).

Patrón 3: "El 2x1" (Proporciones)

Cuándo usarlo: $N(a) = 2N(b)$ o $3k = i$. **Lógica:**

- Opción A (Apilar doble): Por cada 'a' que lees, metes **dos** 'a' en la pila. Luego las 'b' borran de una en una.
- Opción B (Desapilar doble): Metes las 'a' normales. Cuando llegan las 'b', cada 'b' elimina **dos** 'a' de la pila (necesitas un estado intermedio auxiliar para hacer el "doble pop").
- Hay que tener una forma de marcar números negativos también

Patrón 4: "El Multiverso" (La Unión / Ó)

Cuándo usarlo: "Cadenas que cumplen X ó cumplen Y ". **Lógica:** El estado inicial (q_0) no lee nada. Lanza dos transiciones λ (o ϵ) hacia dos caminos distintos.

- Camino 1: Resuelve el problema X .
- Camino 2: Resuelve el problema Y .
- El autómata "adivina" qué camino tomar.

Patrón 5: "Álgebra Simple" (Reorganizar ecuaciones)

Cuándo usarlo: Ecuaciones con restas como $k = i - j$ o $k - i < j$. **Truco:** Los autómatas odian restar, pero aman sumar. Pasa todo a positivo.

- Si te dan $k = i - j \rightarrow$ Transfórmalo en $i = k + j$.
 - Significa: Las 'a' (i) deben ser iguales a la suma de 'b' (j) + 'c' (k).
 - Estrategia: Apila las 'a'. Las 'b' borran 'a'. Las 'c' borran las 'a' que queden.

5.5 Lema del Bombeo para Lenguajes Independientes del Contexto

Sea L un lenguaje independiente del contexto. Entonces existe una constante n (llamada cte. de bombeo) tal que cualquier cadena z perteneciente a L puede descomponerse en 5 partes, $z = uvwxy$, cumpliendo:

1. $|vwx| \leq n$: La "ventana" donde ocurre el bombeo tiene un tamaño limitado.
2. $vx \neq \epsilon$: Al menos una de las dos partes que se bombean (v o x) debe contener algo (no pueden estar ambas vacías).
3. Para todo $k \geq 0$, la cadena $uv^kwx^ky \in L$: Si repetimos v y x el mismo número de veces (k), la cadena resultante sigue perteneciendo al lenguaje.

Si no cumple el lema, no estamos ante un Lenguaje Independiente del Contexto.

Ejemplo:

$$L = \{a^i b^j c^k \mid k = i/j; i, j, k \geq 1\}$$

Y nos dan como componentes (llaman a y z):

- $u = a^{2n-2}$
- $v = a^2$
- $w = \lambda$ (nada)
- $x = b$
- $z = b^{n-1}c^2$

Si usamos $k = 0$ en la expresión uv^kwx^kz , nos queda uwz por lo que tendríamos $a^{2n}b^nc^2$. Además mirando el lenguaje sabemos que se debe cumplir que $N(c) = \frac{N(a)}{N(b)}$. Por lo que podemos sustituir y comprobar si se cumple o no:

$$2 = \frac{2n - 2}{n - 1}$$

Por lo que vemos que cumple el lema (también siguen el orden de abc), no podemos demostrar que no sea un LIC. Y así seguiríamos probando con los k que nos digan. Aunque a simple vista ya se puede afirmar que no lo será porque los LIC no tienen la capacidad de realizar multiplicaciones, solo sumar y contar.

6. Máquinas de Turing

6.1 Concepto Fundamental

Una **Máquina de Turing (MT)** es un modelo matemático de computación que consiste en un autómata con una capacidad de memoria ilimitada en forma de una cinta infinita.

A diferencia de los autómatas finitos o de pila, la MT puede mover su cabezal de lectura/escritura tanto a la izquierda como a la derecha, y puede modificar los símbolos de la cinta.

1. La Cinta (The Tape)

Es la **memoria** de la máquina.

- **Qué es:** Es una tira de papel dividida en casillas (celdas) individuales.
- **Características:** En la teoría, es **infinita** (puedes seguir escribiendo a la derecha o izquierda para siempre).
- **Función:** En cada casilla se guarda un único símbolo del alfabeto (una 'a', un '1', un espacio en blanco...).
- A veces se menciona la "cinta seminfinita", que es una cinta que tiene un muro a la izquierda pero es infinita a la derecha.

2. La Cabeza (The Head)

Es el **lector/escritor**. Es la parte "activa" de la máquina.

- **Qué es:** Un dispositivo que se sitúa sobre una casilla concreta de la cinta en cada momento.
- **Función:**
 1. **Lee** qué símbolo hay en esa casilla.
 2. **Escribe** un símbolo nuevo (borrando el anterior).
 3. **Se mueve** una casilla a la izquierda (*I*), a la derecha (*D*), o se queda quieta, según las instrucciones del estado actual .
- **Analogía:** Es como tu ojo y tu mano con un lápiz, mirando solo un cuadradito de papel a la vez.

3. La Pista (The Track)

Este es el concepto más abstracto y el que suele confundir. Una pista **no es una cinta nueva**, es una **división horizontal** dentro de una misma cinta.

- **Qué es:** Imagina que coges la cinta (que es una fila de casillas) y divides cada casilla horizontalmente en varios pisos o niveles. Cada nivel es una "pista".
- **Para qué sirve:** Permite que la máquina lea o guarde **varios datos a la vez en la misma posición** física.
- **Ejemplo visual:**
 - **Cinta normal (1 pista):** En la casilla 5 hay una "A".
 - **Cinta con 2 pistas:** En la casilla 5, en el *piso de arriba* (pista 1) hay una "A" y en el *piso de abajo* (pista 2) hay un "1".

6.2 Definición

Una MT se define formalmente como $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$.

- Q (**Conjunto de estados**): Los estados finitos internos de la máquina.
- Σ (**Alfabeto de entrada**): Los símbolos que forman la cadena inicial. Es un subconjunto de Γ y **no** incluye el blanco (B).
- Γ (**Alfabeto de la cinta**): Todos los símbolos que pueden aparecer en la cinta. Incluye a Σ y al símbolo blanco.
- δ (**Función de transición**): El "cerebro" de la máquina. Define qué hacer en cada paso.
- q_0 (**Estado inicial**): Donde comienza el cómputo ($q_0 \in Q$).
- B (**Símbolo Blanco**): Representa una celda vacía en la cinta infinita. Pertenece a Γ pero no a Σ .
- F (**Estados finales**): Conjunto de estados de aceptación.

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{I, D\}$$

L(M) significa "**El Lenguaje de la Máquina M**". En términos sencillos: es la lista VIP de palabras que la máquina acepta. La fórmula formal es:

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \alpha_1 q_f \alpha_2, \text{ con } q_f \in F\}$$

1. $w \in \Sigma^*$: Tomas una palabra w cualquiera formada por letras del alfabeto (Σ).
2. $q_0 w$ (**Configuración Inicial**): La máquina empieza en el estado inicial (q_0) con la cabeza lectora al principio de la palabra w .
3. $\vdash^*$ (**Derivación**): Este símbolo significa "después de muchos pasos de cálculo...". Es decir, la máquina lee, escribe, se mueve y cambia de estado muchas veces.
4. $\alpha_1 q_f \alpha_2$ (**Configuración Final**):
 - $q_f \in F$: Lo más importante. La máquina ha llegado a un estado q_f que pertenece al conjunto de estados Finales (F).
 - α_1 y α_2 : Representan "cualquier cosa" que haya quedado escrita en la cinta a la izquierda y a la derecha de la cabeza. A diferencia del ALA, aquí estas cadenas pueden ser enormes, porque la cinta es infinita.

6.3 Funcionamiento y Transiciones

La función de transición se define como $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{I, D\}$. Esto significa que, dado un estado actual y un símbolo leído en la cinta, la máquina realiza tres acciones simultáneas:

1. **Escribe** un nuevo símbolo en la celda actual.
2. **Cambia** a un nuevo estado.
3. **Mueve** la cabeza una posición a la Izquierda (I/L) o a la Derecha (D/R).

Ejemplo de lectura: Si $\delta(q_1, a) = (q_5, b, R)$, significa: "Si estoy en el estado q_1 y leo una 'a', escribo una 'b', paso al estado q_5 y me muevo a la derecha".

6.4 Aceptación y Parada

Una MT opera paso a paso hasta que ocurre una de estas situaciones:

Criterio de Aceptación:

- La MT acepta una cadena w si, partiendo de la configuración inicial, llega a un **estado final** $q_f \in F$.
- **Importante:** A diferencia de los autómatas finitos, **no es necesario leer toda la entrada** para aceptar; basta con alcanzar un estado final.
- Se asume que la máquina se detiene al llegar a un estado final (no hay transiciones definidas desde ahí).

Rechazo y Bucles: Si la cadena no pertenece al lenguaje ($w \notin L(M)$), pueden pasar dos cosas:

1. La máquina se para en un estado **no final** (porque no hay transición definida).
2. La máquina entra en un **bucle infinito** y nunca se para.

NOTA

|Si el lenguaje es...|¿Qué hace con las cadenas que **NO** pertenecen?| ---|---|

|**Recursivo**|**Siempre PARA** (en estado de rechazo). Jamás hace bucle.| |**Rec. Enumerable**|Puede parar (rechazo) **O** entrar en **BUCLE INFINITO**.|

6.5 Protips

Normalmente te recomiendo que dibujes la situación inicial y vayas pensando que necesita hacer tu máquina para llegar al final y después esa idea o razonamiento lo transcribas en forma de autómata.

1. **El "Ping-Pong" (Zig-Zag):** Casi todos los ejercicios se resuelven yendo a buscar algo a la derecha, marcándolo, y volviendo a la izquierda a buscar el siguiente. No intentes hacerlo todo en

una pasada.

- **Tachar, no borrar:** No "elimines" símbolos (dejando huecos en blanco B) a menos que quieras mover toda la cadena. Lo normal es TACHAR (usar el símbolo auxiliar x de tu alfabeto Γ).
 - *Ejemplo:* Para contar un '1', cámbialo por una 'x'. Así sabes que ya lo has contado.
- **La Cinta es tu Memoria:** No tienes variables como `int contador = 0`. Tu contador es lo que hay escrito en la cinta. Si quieres saber si el número es par o impar, tienes que recorrerlo y comprobarlo físicamente.
- **Los Blancos (B) son Muros:** Úsalos para saber dónde empieza y termina tu dato. Si te sales del B , estás en el vacío.

6.6 Variaciones de la Máquina de Turing

6.6.1 Equivalencia General

El punto más importante es que **todas** estas variantes son equivalentes a la MT Estándar . Esto significa que pueden resolver los mismos problemas (computabilidad), aunque la eficiencia (complejidad) varíe.

6.6.2 Variantes de Movimiento y Estructura

Estas variantes cambian cómo se mueve la cabeza o la forma del espacio de trabajo.

- **MT con opción de No-Movimiento:**
 - **Capacidad:** La cabeza puede quedarse estática (E) además de moverse a Izquierda (I) o Derecha (D) .
 - **Simulación:** Una MT estándar simula el "no movimiento" haciendo dos movimientos: uno a la derecha y luego uno inmediatamente a la izquierda (D, I) .
- **MT con Cinta Semi-infinita:**
 - **Estructura:** La cinta tiene un tope a la izquierda, es infinita solo hacia la derecha .
 - **Simulación:** Se usa una MT estándar con **2 pistas** (imaginando que doblamos la cinta infinita por la mitad):
 - Pista superior: Parte derecha de la cinta.
 - Pista inferior: Parte izquierda (en orden inverso) .
 - Se usa un marcador especial (#) para saber cuándo "dar la vuelta" en el extremo .
- **MT Multidimensional (Ej. Bidimensional):**

- **Estructura:** Cinta infinita en más de una dimensión (plano). La cabeza se mueve en 4 direcciones: I , D , AR (arriba), AB (abajo) .
- **Simulación:** Se usa una cinta con **2 pistas**:
 1. Contenido de la celda.
 2. Dirección/Coordenadas asociadas para saber dónde está cada dato .

6.6.3 Variantes de Cintas Múltiples

Estas variantes añaden más cintas o pistas para facilitar el trabajo.

- **MT Multicinta:**

- **Estructura:** Tiene n cintas independientes, cada una con su cabeza .
- **Simulación (¡Muy importante para test!):** Para simular n cintas en una sola, se necesitan $2n$ pistas .
 - Pistas impares: Guardan el contenido de las cintas.
 - Pistas pares: Marcan la posición de las cabezas .

- **MT con Cinta de Entrada:**

- **Estructura:** Una cinta de solo lectura (entrada) + una cinta de trabajo .
- **Simulación:** Se usan **4 pistas**: valores de entrada, posición cabeza entrada, contenido cinta trabajo, posición cabeza trabajo .

6.6.4 Determinismo y Universalidad

- **MT No Determinista (MTN):**

- **Funcionamiento:** La función de transición δ devuelve un **conjunto** de posibles movimientos, no uno solo. Puede "replicarse" para seguir varios caminos .
- **Simulación:** Se crea una cinta con $2n$ pistas, donde n es el número de máquinas a simular. Cada nueva MT implica la inicialización de dos nuevas pistas. Una pista representa el contenido de la cinta y la otra el estado de la MT.

- **Máquina de Turing Universal (MTU):**

- **Concepto:** Es una máquina "reprogramable" que recibe como entrada la descripción de otra máquina M y una cadena w , y simula a M ejecutando w .
- **Estructura:** Típicamente se define con **3 cintas** :
 1. Cinta con la descripción de la máquina a simular (M).
 2. Cinta con el contenido/entrada (w).
 3. Cinta con el estado interno de M .

Tipo de MT	¿Cómo se simula con MT Estándar?	Dato clave para el test
Multicinta (n cintas)	Usa $2n$ pistas	Una pista para símbolo, otra para la cabeza.
Semi-infinita	Usa 2 pistas	Dobla la cinta (pista superior/inferior).
Bidimensional	Usa 2 pistas	Una para contenido, otra para dirección/coords.
Universal (MTU)	Usa 3 cintas	1. Descripción, 2. Contenido, 3. Estado.
No Determinista	Usa $2n$ pistas	No es más potente, solo (teóricamente) más rápida.

6.8 Tesis de Church-Turing

Esta tesis establece una equivalencia entre "algoritmo" y "Máquina de Turing".

- Afirma que cualquier problema que pueda ser resuelto por un algoritmo (computable) puede ser resuelto por una MT.
- No se ha encontrado ningún modelo de computación más potente que la MT.

Un algoritmo para una función $f : D \rightarrow R$ es una *MT*, la cual dada cualquier entrada $d \in D$ en su cinta, finalmente se para con la respuesta correcta $f(d) \in R$ en la cinta:

$$q_0 d \vdash^* q_f f(d), \quad q_f \in F$$

7. Gramáticas GSC Y GSR

7.1 Gramáticas Sin Restricciones (GSR) - Tipo 0

Como su nombre indica, son las gramáticas más flexibles y potentes. No tienen límites en la forma de sus reglas, salvo que siempre debe haber algo que sustituir.

Definición y Reglas Una GSR se define formalmente como $G = (NT, T, S, P)$.

Sus producciones tienen la forma genérica:

$$x \rightarrow y$$

Donde:

- **x (Izquierda):** Es una cadena de símbolos terminales y no terminales (NT/T). La única condición es que **no puede estar vacía** (debe tener al menos un símbolo).
- **y (Derecha):** Es cualquier cadena de símbolos terminales y no terminales. **Puede ser vacía.**

Características Clave

- **Libertad total:** Puedes reemplazar un grupo de símbolos por otro grupo, acortar la cadena, alargarla o borrar símbolos completamente.
- **Lenguaje que generan:** Generan los **Lenguajes Recursivamente Enumerables (LRE)**.
- **Máquina Equivalente:** Son reconocidos por la **Máquina de Turing** estándar.

Ejemplo:

7.2 Gramáticas Sensibles al Contexto (GSC) - Tipo 1

Estas gramáticas imponen restricciones sobre las GSR. La idea principal es que las sustituciones dependen de lo que rodea a la variable ("el contexto") y la cadena nunca se hace más pequeña.

Definición y Reglas Una GSC tiene producciones de la forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

Desglosemos qué significa esto:

1. **A :** Es un **símbolo no terminal** (la variable que vamos a cambiar).
2. **α y β (El Contexto):** Son cadenas de terminales o no terminales (pueden ser vacías).
Representan lo que está a la izquierda y derecha de A .
3. **γ (El Cambio):** Es una cadena **no vacía**. Es por lo que sustituimos a A .

La Regla de Oro: No Contracción La característica más importante para identificar una GSC es la longitud.

- La longitud de la parte derecha (y) debe ser **igual o mayor** que la de la parte izquierda (x).
- Esto significa que la gramática **nunca "encoge"** la cadena (excepto posiblemente para generar la cadena vacía λ si el lenguaje lo permite).

Ejemplo:

Relación con Máquinas

- **Lenguaje que generan:** Lenguajes Sensibles al Contexto (LSC).
- **Máquina Equivalente:** Son reconocidos por los **Autómatas Linealmente Acotados (ALA)**. Es una Máquina de Turing (MT) determinista con su cinta limitada por ambos extremos, el tamaño de la cinta está limitado por el tamaño de la palabra (el tamaño de la cinta no es fijo por sí preguntan en un test). Para que la cabeza lectora no se "caiga" de la cinta, tiene dos símbolos especiales (marcadores) en los extremos izquierdo y derecho que le dicen "hasta aquí puedes llegar" .

La fórmula del Lenguaje $L(M)$. $L(M)$ significa "**El Lenguaje de la Máquina M**". En términos sencillos: es la lista VIP de palabras que la máquina acepta.

$$L(M) = \{w \in \Sigma^+ : q_0[w] \vdash^* [x_1 q_f x_2], q_f \in F \dots\}$$

- **[** (**Marcador izquierdo**): Es el muro de la izquierda.
 - La fórmula $\delta(q_i, [) = (q_j, [, D)$ significa: "Si la cabeza está en el estado q_i y lee el muro **[**, pasa al estado q_j , **deja el muro intacto** (**[**) y se mueve a la **Derecha** (D)".
 - En resumen: **Rebota hacia dentro**. No puede atravesarlo ni borrarlo.
- **]** (**Marcador derecho**): Es el muro de la derecha.
 - La fórmula $\delta(q_i,]) = (q_j,], I)$ significa: "Si lee el muro **]**, **lo deja intacto** y se mueve a la **Izquierda** (I)".
 - En resumen: **Rebota hacia dentro**.

Se traduce así:

1. Tomas una palabra de entrada w .
2. La encierras entre los muros: **[w]**.
3. Empiezas en el estado inicial q_0 .
4. La máquina procesa ($\vdash^*$) moviéndose dentro de esos muros.
5. Si la máquina llega a un estado final ($q_f \in F$) manteniendo los muros intactos ($[x_1 \dots x_2]$), entonces **acepta** la palabra.

Un lenguaje L es sensible al contexto (LSC) si existe una GSC tal que $L = L(G)$ o $L = L(G) \cup \{\lambda\}$.

7.3 Protips

7.3.1 El Mensajero (Commutation)

Problema: Tienes variables mezcladas (ej: $ABAB$) pero necesitas ordenarlas (ej: $AABB$). En una Gramática Independiente del Contexto (CFG) no puedes cambiar el orden una vez generado. En una GSC, sí.

La Estrategia: Creas una regla que permite que dos variables intercambien lugares.

Cómo:

1. **Generación:** Generas pares o tríos desordenados.
2. **Tráfico:** Usas reglas de la forma $XY \rightarrow YX$ para mover la variable "mensajera" a su posición correcta.
3. **Conversión:** Solo cuando están en orden, se convierten en terminales.

Ejemplo Práctico: $L = \{a^n b^n c^n\}$

Queremos generar $aabbcc$.

Intentamos generar bloques ABC .

1. **Regla de Inicio:** $S \rightarrow aSBc \mid abc$ (Esto genera algo como $a(abc)Bc \rightarrow aabcBc$).
 - **Problema:** Tenemos una c antes de una B . El orden está mal: $\dots cBc \dots$
2. **Regla del Mensajero:** $cB \rightarrow Bc$.
 - **Traducción:** Si una c ve una B a su derecha, le dice "Pasa tú primero". La B viaja a la izquierda sobre la c .

Derivación visual:

$$aabc\mathbf{B}c \xrightarrow{cB \rightarrow Bc} aab\mathbf{B}cc$$

Resultado: Ahora las B están con las b y las c con las c .

7.3.2 El Muro (Boundary)

Problema: En las GSC, las reglas son peligrosas. Si tienes la regla $A \rightarrow a$, podrías aplicarla demasiado pronto, antes de que la cadena esté ordenada.

La Estrategia: Usar símbolos especiales (Centinelas) que marcan el principio o el final de la cadena. Las variables no se convierten en terminales hasta que tocan "El Muro", y luego el cambio se propaga

como un efecto dominó.

Cómo funciona:

1. **Colocar el Muro:** Tu regla inicial pone topes. $S \rightarrow \#T\#$.
2. **Contacto:** Una variable solo cambia si toca el muro. $B\# \rightarrow b\#$.
3. **Propagación:** El cambio se contagia de derecha a izquierda (o viceversa). $Ab \rightarrow ab$.

Ejemplo Práctico: Convertir variables a letras ordenadamente

Imagina que tienes la cadena $AAABBB$. Quieres pasarla a minúsculas ($aaabbb$) pero solo si **todo** está listo.

1. Colocamos muro al final: $AAABBB\#$
2. **Regla de Gatillo:** $B\# \rightarrow b\#$ (La última B toca el muro y se transforma).
3. Regla de Dominó: $Bb \rightarrow bb$ (Una B mayúscula ve una b minúscula a su derecha y se transforma).

$$AAABBBb \rightarrow AAABbb$$

4. Regla de Cruce de Frontera: $Ab \rightarrow ab$ (El cambio pasa de las B a las A).

$$AAAbbb \rightarrow AAabbb$$

Por qué es útil: Garantiza que no te queden letras sueltas en medio de variables no procesadas.

7.3.2 El Clonador

Problema: Necesitas duplicar información exacta, como en el lenguaje $L = \{ww\}$ (ej: $abcabc$) o a^{2n} .

La Estrategia: Una variable genera un par (el original y la copia) y una de ellas actúa como "Mensajero" viajando hasta su nueva posición.

Cómo funciona:

Para hacer ww (copiar la palabra exacta):

1. **Generar Pares:** Por cada letra que quieras añadir, generas su par.
 - Si quieres 'a', generas $X_a Y_a$.
2. **Transporte:** Y_a es el clon. Debe viajar saltando sobre las X hasta llegar a la segunda mitad de la palabra.
3. **Regla de Salto:** $Y_a X_b \rightarrow X_b Y_a$ (El clon salta sobre otras letras base).

Ejemplo Práctico: $L = \{ww\}$ con $\Sigma = \{0, 1\}$

Queremos generar 0101.

1. **Semilla:** $S \rightarrow C_0S \mid C_1S \mid \dots$ (Donde C_0 representa el par "Original 0 + Clon 0").
 - Digamos que C_0 en realidad genera X_0Y_0 .
2. **Generación:** Generas $X_0Y_0X_1Y_1$.
 - Aquí tienes "Original 0", "Clon 0", "Original 1", "Clon 1".
 - Orden actual: 0, 0, 1, 1.
 - Orden deseado: 0, 1, 0, 1.
3. **Movimiento (Mensajero):** El Y_0 (Clon 0) debe moverse a la derecha.
 - Regla: $Y_0X_1 \rightarrow X_1Y_0$.
 - Cadena: $X_0\mathbf{Y}_0\mathbf{X}_1Y_1 \Rightarrow X_0\mathbf{X}_1\mathbf{Y}_0Y_1$.
4. **Finalización:** Ahora tienes $X_0X_1Y_0Y_1$ (Grupo 1 separado de Grupo 2). Los conviertes a terminales.

7.4 Identificar Gramáticas y Lenguajes

Gramáticas

¿Qué ves a la IZQUIERDA?	¿Qué ves a la DERECHA?	TIPO
Solo 1 Variable ($A \rightarrow \dots$)	Estricto: a ó aB	Tipo 3 (Regular)
Solo 1 Variable ($A \rightarrow \dots$)	Libre: aAb, BC , etc.	Tipo 2 (GIC)
Grupo ($aA \rightarrow \dots$)	Longitud $\geq$ Izquierda	Tipo 1 (Sensible)
Grupo ($aA \rightarrow \dots$)	Longitud $<$ Izquierda	Tipo 0 (Irrestricta)

Lenguajes

1. Lenguajes Regulares (Tipo 3)

Memoria: Nula o Finitud.

Clave: "No necesito contar hasta el infinito".

- **Lo que SÍ hace:**
 - Acabar/Empezar en algo.
 - Contar módulo fijo (pares, impares, múltiplos de 3).
 - $a^n b^m$ (donde n y m no tienen relación).
- **Lo que NO hace:** Contar indefinidamente y comparar ($a^n b^n$).

2. Lenguajes Independientes del Contexto (Tipo 2)

Memoria: Una Pila (Stack).

Clave: "Puedo comparar DOS cosas (o anidar)".

- **Lo que SÍ hace:**

- **Emparejar DOS grupos:** $a^n b^n$.
- **Desigualdades simples:** $a^n b^p$ donde $p > n$ (La pila sobra, no falta).
- **Espejos/Palíndromos:** ww^R (Lo primero que entra es lo último que sale).
- **Anidamiento:** Paréntesis `(( ))` o `if { if { } }` .

- **Lo que no hace:**

1. **NO puede emparejar TRES grupos:**

- Falla en: $L = \{a^n b^n c^n\}$ o $L = \{a^n b^{n+1} c^{n+2}\}$.
- *Por qué:* La pila se vacía al comparar las `a` con las `b`. Cuando llegan las `c`, ya no te queda memoria de cuánto valía `n`.

2. **NO puede emparejar cruzados (Entrelazados):**

- Falla en: $L = \{0^i 1^j 2^i 3^j\}$ (El famoso $a^n b^m c^n d^m$).
- *Por qué:* Para comparar el 1º con el 3º, tienes que "desapilar" el 2º y lo pierdes.

3. **NO puede hacer COPIAS exactas (generalmente):**

- Falla en: ww (Ej: "mama").
- *Diferencia clave:* ww^R (espejo) es Tipo 2. ww (copia) es Tipo 1.

3. Lenguajes Sensibles al Contexto (Tipo 1)

Memoria: Cinta acotada (puedo leer y volver atrás).

Clave: "Puedo comparar TRES cosas o COPIAR".

Lo que SÍ hace: Todo lo que no podía el Tipo 2.

- **Relaciones Triples:** $a^n b^n c^n$.
- **Dependencias Cruzadas:** $a^n b^m c^n d^m$.
- **Copias Exactas:** ww (Repetir la misma cadena tal cual).

4. Lenguajes Recursivamente Enumerables (Tipo 0)

Memoria: Ordenador completo.

Clave: "Cualquier algoritmo lógico".

- Si te dan un problema lógico complejo que no tiene restricciones de estructura simple. Generalmente, en los exámenes, se centran en los tres anteriores, salvo que pregunten por problemas de parada o indecidibilidad.

8. Decibilidad y Complejidad

8.1 Conceptos Básicos: ¿Qué puede hacer una máquina?

Antes de medir el tiempo, medimos si es *posible* hacerlo.

- **Función Computable:** Una función f es computable en un dominio si existe una MT que computa el valor de f para todos los argumentos del dominio
- **Problema Decidable:** Es un problema cuya respuesta es binaria (**SÍ/NO**).
 - Es decidable si existe una MT da la respuesta correcta (SÍ o NO) para cualquier entrada.
 - Si la máquina se queda en bucle infinito para alguna entrada, el problema NO es decidable.

8.2 El Problema de la Parada (Halting Problem)

El ejemplo clásico de problema **Indecible**.

Planteamiento: Queremos diseñar una "Super-Máquina" H que analice a cualquier otra máquina M con una entrada w y nos diga si M se va a colgar (bucle) o si terminará.

Definición formal de la Máquina H (si existiera): Entrada: w_M (código de la máquina) y w (datos).

- $q_0 w_M w \vdash^* x_1 q_y x_2 \rightarrow$ Si M **se para** con w . (Salida: SÍ)
- $q_0 w_M w \vdash^* x_1 q_n x_2 \rightarrow$ Si M **no se para** con w . (Salida: NO)

Conclusión Teórica:

- **No existe ninguna MT H** que pueda hacer esto para todos los casos.
- Por tanto, el Problema de la Parada es **Indecible**.

Nota importante para test: Si el problema de la parada fuera decidable, todos los Lenguajes Recursivamente Enumerables (LRE) se convertirían automáticamente en Lenguajes Recursivos (LRC). Como no lo es, $LRE \neq LRC$.

8.3 Complejidad Computacional (Medir el coste)

Una vez sabemos que un problema se puede resolver, preguntamos: **¿Cuánto cuesta?**

- **Medida:** Usamos el tamaño del problema (n) y medimos cuánto aumenta el tiempo al crecer n .
- **Notación:** Usamos el orden de magnitud **O(...)** (O grande), no el tiempo exacto en segundos.

- **Tiempo $T(n)$:** Una MT resuelve un problema en tiempo $T(n)$ si no hace más de $T(n)$ movimientos.

Diferencia clave: El Hardware importa

En decidibilidad (teoría pura), todas las MT son iguales. En complejidad (eficiencia), **NO**. El número de cintas cambia la velocidad.

Ejemplo: $L = \{a^n b^n \mid n \geq 1\}$

- **MT Estándar (1 cinta):** Tarda $O(n^2)$. (Tiene que ir y volver muchas veces).
- **MT con 2 cintas:** Tarda $O(n)$. (Puede copiar y comparar en una sola pasada).

8.4 Clases de Complejidad: Determinismo vs No Determinismo

Aquí definimos qué tan difícil es un problema según el tipo de máquina.

Definiciones de Tiempo

- **TD($T(n)$):** Tiempo Determinista. Lo que tarda una MT normal (sin "magia"). Es aceptado por una MT multicinta determinista en tiempo polinómico.
- **TND($T(n)$):** Tiempo No Determinista. Lo que tarda una MT No Determinista (que puede "adivinar" el camino correcto). Es aceptado por una MT multicinta no determinista en tiempo polinómico.
- **Regla:** $TD(T(n)) \subseteq TND(T(n))$. (El determinismo es un caso particular del no determinismo).

Ejemplo: SAT (Satisfacibilidad)

Dada una fórmula lógica (ej: $(x_1 \vee x_2) \wedge \neg x_1$), ¿existe una combinación de True/False que la haga verdadera?

- **MT Determinista:** Tiene que probar todas las combinaciones. Coste exponencial: $O(2^n)$.
- **MT No Determinista:** "Adivina" la combinación correcta al instante y solo verifica. Coste lineal: $O(n)$.

8.5 P vs NP (La jerarquía del mundo real)

Las siglas que definen la informática teórica moderna:

Clase P (Polinomial)

- Lenguajes aceptados por **MT Determinista en tiempo polinómico** ($n, n^2, n^3 \dots$).
- Son los problemas **TRATABLES** (viables de resolver).

- Fórmula: $\bigcup TD(n^i)$

Clase NP (No-Determinista Polinomial)

- Lenguajes aceptados por **MT No Determinista en tiempo polinómico**.
- Son problemas que quizás son difíciles de *resolver* (como SAT), pero muy rápidos de *verificar* si te dan la solución.
- Fórmula: $\bigcup TND(n^i)$

Relación y Tesis

1. $P \subseteq NP$ (Todo problema fácil de resolver es fácil de verificar).
2. **¿P = NP?** La gran pregunta del millón. No se sabe.
3. **Tesis de Cook-Karp:**
 - Clase P = Problemas Tratables.
 - Fuera de P = Problemas Intratables (requieren tanta memoria/tiempo que son inviables para n grande).

8.6 Reducción Polinomial y NP-Completo

¿Cómo clasificamos los problemas más difíciles? Usamos la **Reducción**.

Reducción en tiempo polinomial (L_1 se reduce a L_2)

Significa que podemos transformar cualquier input de L_1 en un input de L_2 rápidamente (tiempo polinómico).

- **Lógica:** Si L_1 se puede transformar en L_2 fácilmente, entonces L_2 es "igual o más difícil" que L_1 .
- **Propiedad Clave:**
 - Si L_1 se reduce a L_2 y $L_2 \in P \Rightarrow$ entonces $L_1 \in P$. (Si el difícil resulta ser fácil, el fácil también lo es).

NP-Completo (Los "Jefes Finales")

Un lenguaje L es **NP-Completo** si cumple dos condiciones:

1. Pertenece a **NP** ($L \in NP$).
2. **Cualquier** otro problema de NP se puede reducir a L .

Es decir, son los problemas más difíciles de toda la clase NP. Si resuelves uno de estos en tiempo polinómico (P), has resuelto todos los problemas de NP (demostrando $P = NP$).